Chapter 5

CS1 course package - University of Nebraska-Lincoln (UNL)

Chris Bourke[1]

[1]University of Nebraska–Lincoln

Abstract

At the University of Nebraska–Lincoln, Parallel and Distributed Computing (PDC) concepts were introduced into multiple sections of a Computer Science I (CS1) course using "codeless" modules consisting of visualizations, simulations, and demonstrations.

These modules are codeless because they do not require students to write or understand code. Instead, students read a short introduction to a PDC concept and then engage with a web-based visualization and/or (code-based) demonstration reinforcing the concept. The codeless nature of these modules makes them suitable for computing and non-computing majors.

In addition, a lab that included distributed computing was updated and modernized while a new distributed programming assignment was created.

Descriptor Elements

Programming Language Employed: C, Java

Relevant core courses: CS1

Pre-requisites: None

Relevant PDC topics:

PDC Concept	Blooms Level
Why and what is parallel or distributed computing?	Remember
Speedup (Parallel and Distributed Models and Complexity	Understand

Learning outcomes:

1. Students should have exposure to parallel, distributed, and asynchronous models of computing and appreciate the potential speed-up these models provide.

Context for use: The University of Nebraska–Lincoln is a large Midwestern R1 institution. The School of Computing is housed in the College of Engineering and offers multiple computing majors (CS, CE, SE, DS, Robotics) and minors and has nearly 1,000 undergraduate computing students. The sections of CS1 in which these modules were deployed include CSCE 155E which uses C as its programming language and mostly consists of CE, CS, and EE majors with an enrollment of about 100; and CSCE 155H an honors section that covers both C and Java in parallel and consists mainly of CS and CE majors who have demonstrated aptitude. Both sections are primarily new freshman students.

These courses are offered in a hybrid manner with scheduled lectures both in-person and livestreamed with recordings available immediately after. Weekly lab and hack sessions are in-person but student may complete their assignments outside of these lab sessions.

5.1 Introduction

Today, large-scale data centers and AI-driven applications are ubiquitous. Underlying these computing paradigms is high-performance computing (HPC), broadly encompassing parallel and distributed computing (PDC). However, many computing graduates do not possess the skills necessary to work effectively in HPC environments. One contributing factor is that computing curricula—particularly at the introductory level—have not kept pace with the realities of modern computing. The last widely adopted single-core processor architectures were introduced well over a decade ago, yet introductory courses are still often taught as if computation occurs in a single-core, single-machine context. Consequently, PDC topics are typically confined to upper-level courses or offered as optional electives, which are among the least frequently taken [3].

There is a clear opportunity to introduce PDC concepts earlier in the curriculum (e.g., CS1/CS2) and to extend this exposure to non-computing majors. Doing so can improve both understanding and appreciation of PDC while enhancing graduates' readiness for the high-performance computing workforce. To address this need, we have developed a series of "codeless" instructional modules designed to introduce foundational PDC concepts to introductory computing students. Our approach is codeless in that it does not require students to write—or even initially understand—code. Instead, the modules consist of web-based interactive visualizations and simulations, making them accessible regardless of programming experience or technical background.

This work builds on a substantial body of research demonstrating that interactive visualizations can significantly enhance student learning outcomes [4, 6, 7]. Our approach also addresses common scalability limitations: the modules are designed to be flexible, enabling integration into a wide range of courses and programming environments, or deployment as standalone, asynchronous learning resources.

In addition, we also detail two additional activities (a lab and a programming assignment) that give students exposure to distributed computing models (cf. Section 5.6).

We did not adopt any unplugged activities as this is a large course that is offered in an asynchronous manner. Our codeless modules are a useful alternative dfor instructors who cannot easily run synchronous, in-class activities due to time, resources, or scale. Instead, these modules can either be used in class as complementary demonstrations to other PDC material or assigned asynchronously.

5.2 How CS1 was changed and PDC topics integrated

The codeless PDC modules were first introduced into two CS1 sections in fall 2024 while the lab and programming assignment were introduced in subsequent years. The codeless modules were assigned to be completed outside of class asynchronously over the period of 8 weeks so that no structural or curricular changes were necessary, making integration easy. The lab was an update of an existing lab while the programming assignment was assigned during the final week of class which did not previously have an assignment.

Table 5.1: Week-to-Week Modules for UNL's CS1

Module	Topics	PDC Notes
1	Course Introduction, Git	
2	Basics: variables, operators	Pre-test data collection
3	Conditionals	PDC Modules assigned
4	Loops	
5	Functions, Unit Testing	
6	Pointers, Pass-by-reference, error handling	
7	Arrays	
8	Debugging, GDB	
9	Strings	
10	File I/O	
11	Encapsulation, structures	
12	Recursion	
13	Searching & Sorting	PDC Modules Due
14	Artificial Intelligence	Post-test data collection

5.2.1 Integrated Syllabi (Pre and Post)

The syllabus for this course is available here. The following learning outcome was added as a result of this project: "Students will have exposure to parallel and distributed computing." No other changes were made. Table 5.1 contains a week-by-week topic list which are organized into modules.

5.2.2 Challenges

The primary challenge was in the development of the modules and material which required a substantial amount of time and effort. However, it was viewed as an investment because the modules, lab, and assignment would be reusable over several years, requiring only minimal maintenance and occasional updates.

If modules are assigned asynchronously, it is recommended that regular reminders are sent out to students on a regular basis with increased frequency closer to the due dates.

5.2.3 Student Time Commitment

Students were given 8 weeks to complete the modules which were estimated to take about 3-4 hours in total. The lab is designed to be completed in 1.25 hours and is typically assigned to be due by the end of the day to accommodate students who do not complete the lab within the lab period. Finally, the assignment had a large variation of completion times because it was more open ended. Some students completed it extremely quickly if the GenAI was able to generate a complete solution. Others took longer if the GenAI was not as efficient or produced buggy code. Yet others took even longer because they wanted to improve their game and add additional features that were not required. This is summarized in Table 5.2.

Table 5.2: Time Commitment for CS1 Activities - UNL

Activity	Typical Student Time
Module 1.0	2.0 hours
Module 2.0	0.5 hours
Module 3.0	1.0 hours
Encapsulation Lab (USGS)	1.25 hours
Vibe Coding (Battleship)	2 - 5 hours

5.3 New Unplugged Activities

No unplugged activities were developed or adopted for this course. Enrollment is typically large and the hybrid nature of the course does not readily lend itself to in-class unplugged activities.

5.4 New Plugged-in Activities

5.4.1 Codeless PDC Modules

Each codeless module is structured as follows: students are asked to read a short introductory text explaining the concepts which will be demonstrated through simple browser-based activities. They then are asked to engage in interactive visualizations or simulations which do not require them to develop or understand any code. Some of the modules include an additional demonstration which requires the student to compile and run a program but does not require them to write, edit, or read any code. Finally, students are asked to reflect on the concepts by answering a series of questions.

5.4.2 Structure

The reading material has been written at a level that does not require a deep understanding of PDC concepts. The material deliberately omits many details that would be necessary to gain expertise, and instead favors describing concepts in a more abstract manner which should make them more accessible to a wider audience. For example, we do not distinguish between data-parallel and task-parallel models, nor do we provide rigorous definitions of cores, threads, or processes. However, we do mention these concepts in a way that early computing students should be able to relate to and understand.

The simulations include both web-based visualizations and pre-written, code-based simulations. The web-based visualizations are interactive applications inspired by the highly successful utilization of similar technologies introduced to visualize programs [7], data structures and algorithms [6], and automata [4]. They can be run on any web browser without special environments or software installations. The code-based simulations include both C and Java versions to accommodate students in different introductory courses. However, we still describe them as "codeless" because students are not required to write or design any code. They are required to compile and run the programs, but we include complete, minimal

instructions similar to what they should have already seen in other classes with programming assignments. It is unavoidable to require a proper C or Java environment for compiling and running these simulations, but we have tried to minimize external dependencies as much as possible.

At the end of each module, students are prompted to reflect on the reading and their interaction with the browser-based simulation and demonstration by answering a series of questions designed to ensure that: 1) they ran a sufficient number of use cases in each simulation; and 2) their simulation experience directly traces back to the reading which is intended to reinforce understanding. For example, they are asked how much time a simulation took to execute with different numbers of threads and are then asked to deduce specific reasons for trends they saw (*i.e.*, seeing a roughly double speed-up when doubling the threads, but not seeing an improvement beyond the number of cores available on the system). To help understanding and avoid misconceptions, suggested answers are provided based on our own simulations. This reflects our intent for the modules to be a formative, rather than summative, assessment.

Module 1.0

The first module introduces students to asynchronous and parallel computing through several web-based visualizations which contrast with sequential and synchronous computing. The consequences of each type of computing are highlighted in the visualizations and demonstrations. In one example, students are given a browser-based GUI which executes a synchronous, blocking action that freezes the UI, and an asynchronous action which does not.

The main demonstration is a parallel computing simulation and visualization in which the user can specify a number of generic computing tasks, a number of processors to execute them, and how long each task will take (*e.g.*, 2-6 seconds). The simulation then queues the tasks for each processor to execute in order. The simulation reports the status of each task as well as total wall-clock time compared to processor time. Figure 5.1 contains an example screenshot of the visualization.

Module 2.0

The second module introduces students to the concept of threads. It is based on a brute-force, dictionary-based, password-cracking scheme which uses an "embarrassingly parallel" approach. We give students a basic explanation how password hashing works to motivate the activity as well as provide early exposure to security issues such as the importance of strong passwords. They are provided both sequential and parallel password-cracking programs with a C version using `pthreads` and a Java version using `Callables`. We include full instructions on cloning, compiling, and running the programs, as well as how to read and interpret the results. No actual coding is necessary to run and observe the simulations.

The demonstration exposes students to the potential performance improvements of parallel computation and its limits. They observe (roughly) proportional speed ups until the number of threads exceeds the number of processors after which no further improvement is observed. When they run the program, they can specify how many threads to create. They

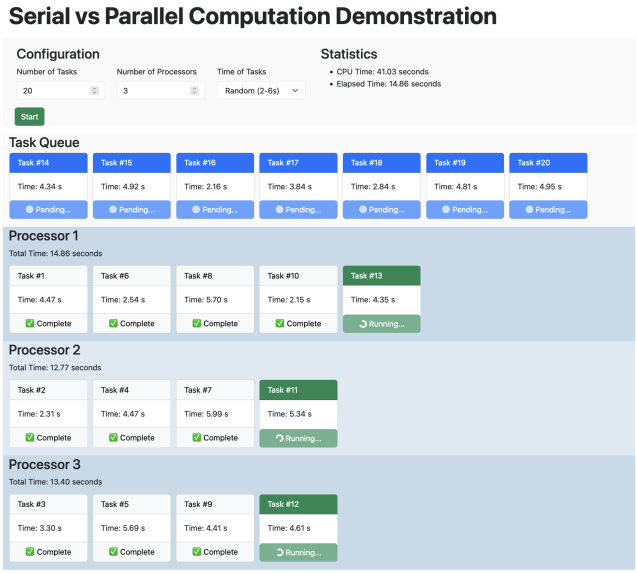

Figure 5.1: A screenshot of Module 1.0's Parallel Computing Demo

are instructed to run the program multiple times with 1, 2, 4, *etc.* threads and observe the trends. An example run of the C version's output can be found in Figure 5.2.

We note that the demonstration is data-parallel because the dictionary listings are split evenly among each thread, so we chose password hashes which ensures the program does not get "lucky" by finding the hash early in any particular data slice, regardless of how many threads are specified.

Module 3.0

The final module introduces students to a producer-consumer pattern. Students are provided another browser-based visualization as well as a C and Java implementation to investigate. In all versions, generic tasks represent requests or work which are simulated through HTTPS requests to a web-service that causes a delay before responding. The length of the delay can be specified or randomized.

All of the simulations allow students to specify the number of producers and of consumers. Once started, each producer will generate tasks at random intervals and place them into a task queue for processing. When a consumer is available, it takes the next available task from the queue and executes it. The simulation will continue until stopped by the user. An example screenshot of the web-based visualization can be found in Figure 5.3.

As with the other modules, the questions point students to observe several scenarios, such as having equal or unbalanced numbers of producers or consumers. Students are then asked to predict the consequences of those different scenarios.

```
creating 4 threads...DONE
Starting threads...
Starting thread 1...
Starting thread 2...
Thread 2 hashing [61851, 123702) = [down, mallorca]...
Starting thread 3...
Thread 1 hashing [0, 61851) = [a, downscaled]...
Starting thread 4...
Thread 3 hashing [123702, 185553) = [mallorcan, rotc]...
Thread 4 hashing [185553, 247406) = [rotch, zzz]...
Thread 1 did not crack password
Thread 4 cracked password: 0x618e0fcd...b0b => zzz42
Signaling other threads to terminate...
All threads DONE
Time:
218.541u 0.015s 0:54.82 398.6% 0+0k 4888+0io 0pf+0w
```

Figure 5.2: Example run of parallel password cracking. The contents of a dictionary are split among a specified number of threads ("down" to "mallorca" for example).

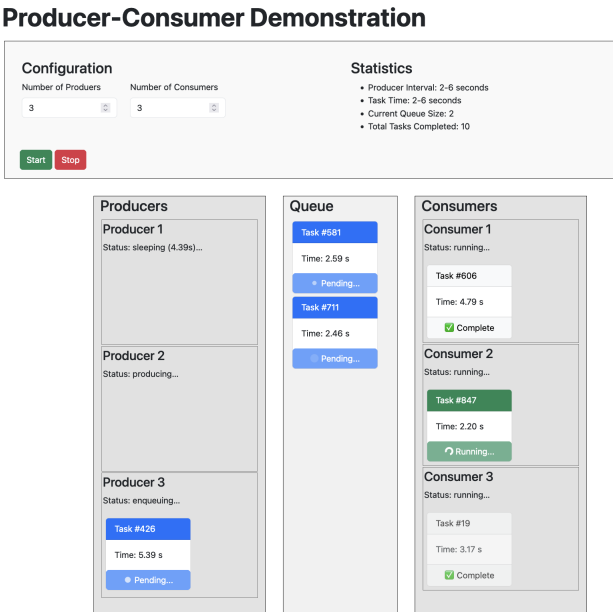

Figure 5.3: A screenshot of Module 3.0's Producer-Consumer Demo

5.5 Course-wide Pre and Post Course Surveys

To determine if our codeless modules were effective at teaching basic PDC concepts, we administered pre and post exams and surveys.

To measure students' knowledge and understanding of PDC concepts, they were asked to provide free-form text answers to the following questions. For each of the four identified PDC concepts: sequential (S), asynchronous (A), parallel (P), distributed (D) computing, students were asked to provide:

1) a definition (D) of the concept; and

2) an example (E) of the concept

To measure attitudes, students were first asked to agree or disagree with the following statements:

A1 In general, my interest in computing is high

A2 My interest in learning about parallel and distributed computing is high

A3 In general, I believe learning about parallel and distributed computing is important

A4 I believe I have a good understanding of computing

A5 I believe I have a good understanding of parallel and distributed computing

Student answers were collected using a 7-point Likert scale (7 = strongly agree, ... 4 = neutral, ... 1 = strongly disagree).

Results were very strong. Across all dimensions, our codeless PDC modules showed statistically significant and substantial (with respect to effect size) improvement in understanding and appreciation of PDC. For a complete analysis see [2] and [1].

In addition, we administered the common pre/post survey (see Section 1.5); the results can be found in Table 5.3. Computing background data indicated a negligible to small improvement which reflected the already-high ratings students self-reported. This is expected as this population are computing majors and would already have a substantial computing background. The programming language familiarity was as expected. The main source of data was from courses that used C as the introductory language and so it showed a large improvement in familiarity while other languages did not. Likewise, the computing concepts showed medium improvements overall. Some of the concepts only showed a small improvement but this is expected as classes and objects are not directly covered when using C (we cover structures and encapsulation but do not use this terminology). Finally, the results of the PDC concepts were consistent with the observations published in [2] and [1]. Students showed substantial improvements in familiarity with respect to parallel and distributed computing.

5.6 Other Activities and Ideas

In addition to our codeless PDC modules, we introduced both a lab and a programming activity to give students exposure to distributed computing.

5.6.1 Lab: Processing a Remote Data Source

A lab on encapsulation (designing and implementing a C structure) had been a staple of this course for over 10 years. Previously, it involved connecting to various RSS feeds and displaying the most recent articles. However, in recent years RSS's popularity has declined. This gave us motivation to update this lab to instead process a live JSON data feed based on the Earthquake Tracker Remote Data Retrieval Assignment (see Section 1.4.4).

Students are not expected to write code to connect to the server and process the JSON responses directly. For our version, starter code is provided to do so and students then design and implement a structure to model the earthquake data. Students are also required to process the data to report the closest earthquake, strongest earthquake, etc.

5.6.2 Programming: Vibe Coding a Client-Server Battleship Game

As part of a larger effort to introduce AI and vibe coding into CS1, we developed a vibe coding activity that had students develop a client program to connect to a remote server to play a game of battleship. In this version, the client makes "moves" by submitting JSON requests to a remote server (calling "shots" on a 10×10 grid). The server responds with JSON data indicating a hit or miss and an updated score (each game has a limited number of shots) and game state (playing, win, loss, etc.).

The activity introduces students to distributed computing and network communication and requires formatting JSON data, performing HTTP requests and processing JSON responses in C. This would generally be considered to be too advanced even at the end of a CS1 course, but with the aid of an LLM, it is feasible. Students are introduced how to use an LLM, common pitfalls, and how to prompt the LLM, test the code it produces and then follow up to refine and debug its code.

In our first offering of this activity, over 75% of the students created an interactive client that was able to successfully communicate with the server and win a game of battleship.

5.7 Conclusion

While most PDC efforts have focused on programming strategies for upper-level and elective courses, we believe there is value and opportunity in introducing these concepts much earlier in the computing curriculum. Our "codeless" approach is a simple, accessible, and scalable way to introduce PDC concepts to both computing majors and non-majors as early as CS0. Our studies have demonstrated that our approach has statistically significant and substantial gains in both learning and appreciation for PDC topics.

For instructors thinking about developing similar content, advanced planning is highly advised. The codeless PDC modules are perfect for asynchronous delivery and take minimal class time. However, developing the content required a substantial upfront investment in planning and time. However, once the modules were created, minimal maintenance was required.

Instructors have reported that it is easy to adopt the codeless modules as in-class demonstrations to complement their own PDC content coverage. The modular nature means that

material can be adopted in part or in whole or remixed with other content. Using our modules in this manner does not require any additional development time and so this path would be ideal for instructors.

If instructors assign modules outside of class, you can expect students to complete all the modules within 3-4 hours total (less if only parts are assigned). One challenge may be to ensure that students complete the modules asynchronously. This can be addressed by adopting them as part of in-class activities or by creating small assessments to complement the modules.

Table 5.3: Survey Results for CS1 - UNL. Sample size was 125 and included data from fall 2024 and fall 2025 semesters. The pre- and post means are reported along with the difference (Diff). Statistical significance is measured using a non-parametric Wilcoxon signed-rank (paired) test. The Wilcoxon statistic (W) is reported along with the p-value. Significance is indicated as Yes using $p < 0.05$. Effect size is reported using Cliff's Delta (δ). A negative value indicates an increase in the post-survey data. The interpretation of the effect size is reported (negligible, small, medium, large) as suggested by [5].

Item	Pre	Post	Diff	W	p-value	Sig.	Cliff's δ	Interp.
Computing Background								
Computer Use	5.80	5.84	0.04	884.5	0.643	No	-0.04	negligible
Programming	3.61	4.14	0.53	567.5	0.000	Yes	-0.21	small
Programming Compared to Peers	3.53	3.97	0.44	866.0	0.000	Yes	-0.17	small
Programming Languages								
C	2.50	4.19	1.70	164.5	0.000	Yes	-0.68	large
C++	1.54	1.97	0.42	286.0	0.000	Yes	-0.21	small
C#	1.42	1.66	0.24	104.0	0.001	Yes	-0.11	negligible
Java	2.70	2.84	0.14	327.0	0.101	No	-0.02	negligible
JavaScript	2.26	2.26	0.00	466.0	0.928	No	-0.01	negligible
PHP	1.13	1.23	0.10	8.0	0.022	Yes	-0.04	negligible
Python	2.74	2.90	0.16	610.0	0.074	No	-0.07	negligible
Scratch	2.06	2.18	0.12	341.5	0.158	No	-0.07	negligible
Visual Basic	1.15	1.20	0.05	47.0	0.264	No	-0.04	negligible
other	1.82	1.85	0.02	540.0	0.995	No	-0.02	negligible
Computing Concepts								
Data/Variable Types	4.34	5.40	1.06	160.0	0.000	Yes	-0.33	medium
Arithmetic Expressions and Operators	4.35	5.22	0.86	266.5	0.000	Yes	-0.25	small
Branching	4.11	5.21	1.10	230.5	0.000	Yes	-0.32	small
Loops	3.97	5.21	1.24	186.0	0.000	Yes	-0.35	medium
Calling functions/methods	3.76	4.99	1.23	273.0	0.000	Yes	-0.35	medium
Arrays	3.13	4.61	1.48	375.0	0.000	Yes	-0.44	medium
Writing static functions/methods	2.95	4.34	1.38	354.0	0.000	Yes	-0.43	medium
Writing functions/methods	3.22	4.70	1.48	224.5	0.000	Yes	-0.44	medium
Writing classes	2.70	3.62	0.92	433.5	0.000	Yes	-0.32	small
Creating objects	2.62	3.48	0.86	422.0	0.000	Yes	-0.31	small
PDC Concepts								
Parallel Computing	1.66	2.70	1.05	526.5	0.000	Yes	-0.55	large
Distributed Computing	2.69	3.07	0.38	983.5	0.007	Yes	-0.17	small
Event Handling	1.76	2.68	0.92	483.5	0.000	Yes	-0.44	medium

Bibliography

[1] Chris Bourke. Codeless modules for parallel and distributed computing in early computing curriculum. In Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1, pages 141–147, New York, NY, USA, 2026. Association for Computing Machinery.

[2] Chris Bourke and Justin Firestone. Codeless PDC modules for early computing curriculum. In 2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pages 357–364, 2024.

[3] Steven Bradley, Miranda C. Parker, Rukiye Altin, Lecia Barker, Sara Hooshangi, Thom Kunkeler, Ruth G. Lennon, Fiona McNeill, Julià Minguillón, Jack Parkinson, Svetlana Peltsverger, and Naaz Sibia. Modeling women's elective choices in computing. In Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education, ITiCSE–WGR '23, page 196–226, New York, NY, USA, 2023. Association for Computing Machinery.

[4] Susan Rodger. Learning automata and formal languages interactively with jflap. SIGCSE Bull., 38(3):360, jun 2006.

[5] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek. Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the NSSE and other surveys. Presented at the annual meeting of the Florida Association of Institutional Research, 177, 2006.

[6] Clifford A. Shaffer, Matthew L. Cooper, Alexander Joel D. Alon, Monika Akbar, Michael Stewart, Sean Ponce, and Stephen H. Edwards. Algorithm visualization: The state of the field. ACM Trans. Comput. Educ., 10(3), aug 2010.

[7] Jaime Urquiza-Fuentes and J. Ángel Velázquez-Iturbide. A survey of successful evaluations of program visualization and algorithm animation systems. ACM Trans. Comput. Educ., 9(2), jun 2009.

Appendix

Github Link to Instructional Material

Codeless Modules

- GitHub source: `https://github.com/cbourke/PDCatUNL`
- Live demo: `https://go.unl.edu/PDCatUNL`

Labs

- Distributed computing & encapsulation lab (processing USGS earthquake data: `https://github.com/cbourke/CSCE155-C-Lab11`
- Distributed computing & Vibe Coding Hack (battleship game client): `https://github.com/cbourke/ComputerScienceI/tree/master/hacks/hack14.0`

Detailed Pre and Post Syllabus

See Section 5.2.1

Organization of Material

- The Codeless repository is a webapp Eclipse project primarily written in JavaScript with a Java backend. It also contains a `c` folder for all C code.
- Both labs are C projects that are framework agnostic and can be directly cloned. The setup does require 'apt-get' or similar package manager for dependendies.

Survey and Other Anonymized Data and Analysis

No data is shareable as per our Institutional Review Board. For analysis see Section 5.5